

Homework 05: An Object That Cannot Be Broken

Build a class whose rules are enforced by the compiler and the constructor rather than by a comment asking callers to be careful. Then try to break it and show that you cannot.

The Task

Create `Booking.java` in the classic form: a `public class Booking` with a `public static void main(String[] args)`.

Inside it, write a `Reservation` class holding:

- a passenger name
- a row number
- a list of seat labels, such as `["A", "B"]`

The Rules It Must Guarantee

A `Reservation` may never exist in a state that breaks any of these:

1. The passenger name is never null and never blank
2. The row is always 1 or greater
3. There is always at least one seat
4. Its seat list can never be changed by anyone outside the object, **including whoever passed the list in**

Rule 4 is the interesting one, and it has two halves. Think about what the caller still holds after they call your constructor.

Prove It

In `main`, attempt each of these and print what happens:

- a null passenger
- a blank passenger
- row `0`
- an empty seat list
- keeping a reference to the list you passed in, then adding to it afterwards
- calling `add` on the list your getter hands back

All six attempts must fail or have no effect. Your output should make it obvious that the object defended itself each time.

Also Include

- At least one static factory method, plus a comment saying what it gives you that a constructor does not
- A `toString()` override so printing a reservation is readable

Acceptance Criteria

- ☐ Runs with `java Booking.java`
- ☐ `public class Booking` with `public static void main(String[] args)`
- ☐ Every field in `Reservation` is `private` and `final`
- ☐ All four rules are enforced in the constructor, before any field is assigned
- ☐ Mutating the caller's original list after construction does not change the reservation
- ☐ Mutating the list returned by the getter either fails or does not change the reservation
- ☐ A static factory exists, with a comment justifying it
- ☐ `toString()` is overridden

Stretch

Make the seat list a plain `ArrayList` field instead of an immutable one, and get rule 4 working anyway. Then write a comment comparing the two approaches: what does each cost, and on which side does the cost fall, the constructor or every call to the getter?

Hint, if rule 4 is slipping through

There are two separate leaks, and fixing one does not fix the other.

The first: after `new Reservation(..., seats)`, who else still holds a reference to that exact list object?

The second: after `reservation.seats()`, what did you just hand out?

`List.copyOf` addresses both at once, and working out why is the point.